

Sentencias de Control en C++

Profesor Dr. Paul Bustamante

Índice

- 1. Introducción**
- 2. Bifurcaciones: If, else, else-if y switch**
- 3. Bucles: for, while, do-while**
- 4. Break y continue**
- 5. Ejemplos**

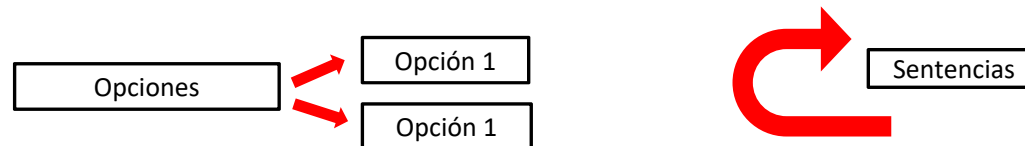
Introducción

Las **sentencias** de un programa en C++ **se ejecutan *secuencialmente***, esto es, cada una a continuación de la anterior, empezando por la primera y acabando por la última.

Cada sentencia termina con un **Punto y coma**.

Para poder modificar el flujo secuencial de la ejecución, el C++ dispone de varias sentencias, de las cuales, las más utilizadas, se pueden agrupar en dos familias:

1. **Sentencias de Bifurcación:** Permiten elegir entre dos o más opciones según ciertas condiciones: *if, else, else if y switch*.



2. **Sentencias de Iteración o Bucles:** Permiten ejecutar repetidamente un conjunto de instrucciones tantas veces como se desee, cambiando o actualizando ciertos valores: *for, while y do-while*.
 - También existen las sentencias *break, continue y goto* que trabajan en conjunto con las anteriores.

En las sentencias de control se usan mayoritariamente los **operadores relacionales** y lógicos, para la evaluación de las expresiones.

Recordatorio: operadores

Operadores Relacionales		
Operador	Utilización	Devuelve true si:
>	op1 > op2	op1 es mayor que op2
>=	op1 >= op2	op1 mayor o igual que op2
<	op1 < op2	op1 es menor que op2
<=	op1 <= op2	op1 es menor o igual que op2
==	op1 == op2	op1 es igual a op2
!=	op1 != op2	op1 distinto de op2

Ejemplo:

```
int x1=9, x2=3, res;           //9=00001001  3=00000011
res=(x1>x2) ? 10:15;           // res = 10
res =(x1<x2) ? 10:15;           // res = 15
res = x1++;                     //res = 9 , x1 = 10
                                //Pasos: 1)asigna 2)incrementa
res = x1;                       //res = 10
res = ++x2;                     //res=4 , x2 = 4
                                //Pasos: 1)incrementa 2)asigna
```

2. Bifurcaciones: if, else y else if

- ❑ La **sentencia if** permite ejecutar una sentencia simple o compuesta según se cumpla o no una determinada condición.

Ejemplo: Forma 1: **if** (expresion) sentencia;
 Forma 2: **if** (expresion) {sentencia1; sentencia2;}

- ❑ La **sentencia else** se usa junto con la sentencia **if**. Permite realizar una bifurcación ejecutando una parte u otra del programa, según se cumpla o no la condición:

Ejemplo: **if** (expresion) {sentencia1; sentencia2;}
 else {sentencia3; sentencia4;}

Explicación: se evalúa la expresión. Si el resultado es **true**, se ejecutan las *sentencias 1 y 2*. Si el resultado es **false**, se ejecutan las *sentencias 3 y 4*.

- ❑ La **sentencia if - else if** permite realizar una ramificación múltiple, ejecutando *una* entre varias partes del programa, según se cumpla *una* entre *n* condiciones.

Ejemplo: **if** (expresion1) sentencia1;
 else if (expresion2) sentencia2;
 else if (expresion3) { sentencia3_1; sentencia3_2; }
 else sentencia4;

Explicación: Se evalúa *expresion1*. Si el resultado es **true**, se ejecuta *sentencia1* y sale de este bloque de **if-elseif**. Si el resultado es **false**, pasa a evaluar *expresion2* y así sucesivamente. Si todas las expresiones son **false**, se ejecutará la *sentencia4*.

2. Bifurcaciones: `switch`

Esta sentencia desarrolla una función similar a la de las sentencias *if..else* de múltiples ramificaciones, aunque presenta importantes diferencias.

El **formato general** de esta sentencia es:

```
switch(expression){  
    case valor1: sentencia1; break;  
    case valor2: sentencia2; break;  
    case valor3: sentencia3; break;  
    default: sentencia4;  
}
```

Ejemplo

```
int tecla;  
cin >> tecla;  
switch(tecla){  
    case 1: Abrir(); break;  
    case 2: Guardar(); break;  
    case 3: Salir(); break;  
    default: Error();  
}
```

La sentencia ***switch*** se usa en conjunto con las sentencias *case*, *default* y *break*.

Características:

- Cada sentencia **case** se corresponde con un único valor de expresión . No se pueden establecer rangos o condiciones.
- Si una sentencia case no lleva la sentencia **break** al final, el control pasa a la sentencia case siguiente, aunque su valor no corresponda con expresión.
- Los valores no comprendidos en las sentencias case se gestionan en default, que es opcional.

3. Bucles: for

El bucle es un bloque donde se puede ejecutar repetidamente un número determinado (o indeterminado) de veces unas líneas de código.

La sentencia *for* es el bucle más versátil y utilizado en C++. Su **formato general** es el siguiente:

```
for (inicialización ; expresion_de_control ; actualizacion){
    Sentencia1;
    Sentencia2;}
```

Donde:

- En *inicialización* se inicializan las variables que intervienen en el bucle for. **Sólo se hace una vez, al inicio del bucle.**
- La *expresion_de_control* es la que nos va a permitir continuar o terminar el bucle.
- En *actualizacion* se les da los nuevos valores a las variables, antes de que se ejecute la *expresion de control*.
- La *sentencia1* y *sentencia2* se ejecutarán tantas veces cómo lo permita la *expresion_de_control*.

Ejemplo

```
for (int Num = 0 ; Num < 3 ; Num++) {
    cout << "Num=" << Num << endl;
}
```

$\gg$

```
Num=0
Num=1
Num=2
```

Ejemplo_2_1

```
for (int cont=0, x=10, Num=1; Num<4; Num++, cont+=2) {
    x = x*Num;
    cout << "x=" << x << " cont=" << cont << endl;
}
```

$\gg$

```
x=10 cont=0
x=20 cont=2
x=60 cont=4
```

3. Bucles: `while`

Esta sentencia permite ejecutar repetidamente, *mientras se cumpla una condición*, una sentencia o bloque de sentencias.

Su **forma general** es la siguiente:

```
while (expresion_de_control){  
    Sentencia1;  
    sentencia2;  
}
```

La *Sentencia1* y *Sentencia2* se ejecutarán repetidamente hasta que se cumpla la condición que hay en la *expresion_de_control* (la cual es una expresión booleana o lógica, true o false).

Ejemplo: ¿Cómo hacer que un programa esté continuamente ejecutándose?

Ejemplo_2_2

```
void main (){  
    int opt = 1;  
    while(opc==1){  
        ...  
        ...  
        if (expresion_de_control) break;  
    }  
}
```


3. Bucles: do - while

Esta sentencia funciona de modo análogo a **while**, con la diferencia de que **la evaluación se realiza al final del bloque** (en la sentencia **while**), de manera que las sentencias se ejecutan por lo menos una vez.

Su forma general es:

```
do {  
    Sentencia1;  
    sentencia2;  
} while (expresion_de_control);
```

Ejemplo: ¿Cuál es la diferencia entre los dos códigos?

```
int Num=1;  
while(Num <= 3){  
    cout << "Num=" << Num << endl;  
    Num++;    //equivale a Num=Num+1;  
}
```

```
int Num=1;  
do{  
    cout << "Num=" << Num << endl;  
    Num++;    //equivale a Num=Num+1;  
}while(Num <= 3);
```

Ejemplo_2_3

¿Y si comenzáramos con un valor de: **int Num = 4;**

3. Bucles: break, continue y goto

Sentencia `break`: esta instrucción **interrumpe la ejecución del bucle** donde se ha incluido, haciendo al programa salir de él, aunque la *expresion_de_control* no se haya cumplido.

Sentencia `continue`: esta sentencia **hace que el programa comience el siguiente ciclo del bucle** donde se halla, aunque no haya llegado el final de las sentencias o bloque.

Sentencia `goto`: permite saltar a una determinada etiqueta del programa. **No se recomienda su uso** debido a que disminuye la legibilidad y claridad del código. Sólo se mantiene por motivos de compatibilidad.

Ejemplo:

```
int Num=1;
while(true){
    cout << endl << "Numero:";
    cin >> Num;
    if ( Num == 0) break; //termina
}
```

?
=

```
int Num=1;
while(Num!=0){
    cout << endl << "Numero:";
    cin >> Num;
}
```

```
for (int Num=1;Num<30;Num++){
    if (Num%2) continue;
    //Num%2=0 para pares, 1 impares
    cout << "Num: " << Num << endl;
}
```

>>

```
Num:2
Num:4
Num:6
...
Num:28
```

Ejemplo_2_4

4. Ejemplos

Ejemplo: *Conversor de Temperatura mejorado*

```
#include <iostream>
using namespace std;
#include <conio.h> //para getch()
void main (){
    int opc=1; double temp1,temp2; //declaración de variables
    while(true) {
        system("cls");
        cout << "Conversor de Temperatura" << endl;
        cout << "-----" << endl;
        cout << "0: Salir." << endl;
        cout << "1: de gCent. a gFarenh." << endl;
        cout << "2: de gFarenh. a gCent." << endl;
        cout << "\tOpcion: ? ";
        cin >> opc;
        if (opc == 0) break; //Salir del while
        if (opc<1 || opc>2) continue; //También: if (opc!=1 && opc
!= 2)
            cout << "Dar Temperatura: "; cin >> temp1;
        switch(opc){
            case 1: temp2 = 9.0/5.0*temp1 + 32;
                    cout << temp1 << "gF es " << temp2 <<"gC"<< endl; break;
            case 2: temp2 = (temp1-32)*5.0/9.0;
                    cout << temp1 << "gC es " << temp2 <<"gF"<< endl;
                    break;
        }
        cout << "\tPresione una tecla para continuar.."<<endl;
        getch(); //lee una tecla
    }
    cout << "Fin.." << endl << endl;
}
```

Ejemplo_2_5

4. Ejemplos

Ejemplo: Genera números aleatorios entre 0 y 1

```
#include <iostream>
using namespace std;
#include <stdlib.h>    //para rand()

#define NUM 20

void main ()
{
    char ch=1;
    while(ch!=88){
        for (int i=0;i<NUM;i++){
            double tmp=(double)rand()/32767.0;
            cout << tmp << endl;
        }
        cout << "'X' para salir";
        cin >> ch;
        system("cls");
    }
}
```

Ejemplo_2_6

Ejemplo: Genera números aleatorios entre dos valores dados

```
#include <iostream>
using namespace std;
#include <stdlib.h>
//rand() da un valor máximo de 32767

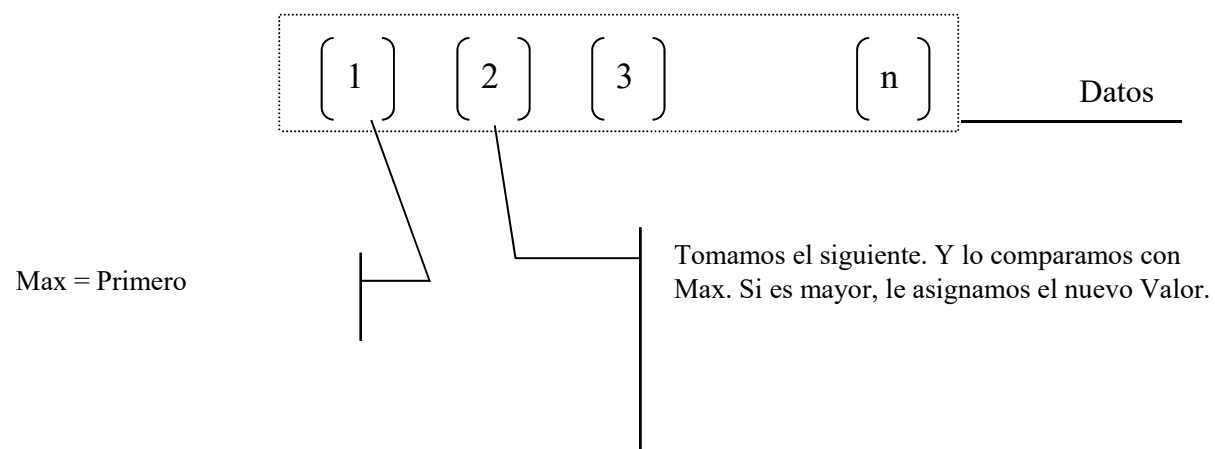
void main ()
{
    //generador aleatorio de números
    int Num;
    int LimInf, LimSup;
    cout << "Cuantos num. desea generar?";
    cin >> Num;
    cout << "Limite inferior y superior?";
    cin >> LimInf >> LimSup;
    for (int i=0;i<Num;i++){
        double tmp=(double)rand();
        double n = (LimSup-LimInf)*tmp/32767.0 + LimInf;
        cout << n << endl;
    }
}
```

Ejemplo_2_7

4. Ejemplos

Ejemplo:

Calculando el mayor de varios números



Ejemplo_2_8